

Học máy: Cây Quyết định Nâng cao và Phương pháp Kết hợp

Quoc Kien

1. Cơ chế Hoạt động của Cây Quyết định (Decision Tree)

Bản đồ ký hiệu khi tính split

Ký hiệu	Nghĩa	Cách dùng
S	Tập dữ liệu tại node hiện tại	Tính impurity cha.
A	Thuộc tính/điều kiện đang xét để chia	Thử từng thuộc tính hoặc từng ngưỡng.
S_v	Nhánh con khi thuộc tính nhận giá trị v	Tính impurity con.
p_c	Tỉ lệ mẫu thuộc lớp c	Dùng trong entropy/Gini.
$H(S)$	Entropy	Đo độ lẫn lộn bằng log.
$Gini(S)$	Gini impurity	Đo độ lẫn lộn không dùng log.
$IG(S, A)$	Information Gain	Impurity cha trừ impurity con có trọng số.
SSE	Tổng bình phương sai số trong regression tree	Chọn split làm SSE nhỏ nhất.

Luồng làm bài: tính impurity của node cha, chia dữ liệu theo từng ứng viên split, tính impurity/SSE của từng nhánh, lấy trung bình có trọng số, rồi chọn split làm giảm lỗi nhiều nhất.

Cây quyết định là thuật toán học máy phi tuyến tính hoạt động theo cơ chế chia để trị (Top-down, Greedy, Recursively partitioning). Thuật toán liên tục phân tách không gian thuộc tính thành các vùng hình chữ nhật loại trừ lẫn nhau (Mutually exclusive regions) sao cho dữ liệu ở mỗi vùng con đạt độ đồng nhất (Purity) cao nhất.

Tại một nút (node) bất kỳ, không gian dữ liệu được phân hoạch dựa trên một thuộc tính j và một ngưỡng giá trị t :

$$S(j, t) = \{\{x \in \mathcal{D} \mid x_j \leq t\}, \{x \in \mathcal{D} \mid x_j > t\}\} = \{R_1, R_2\}$$

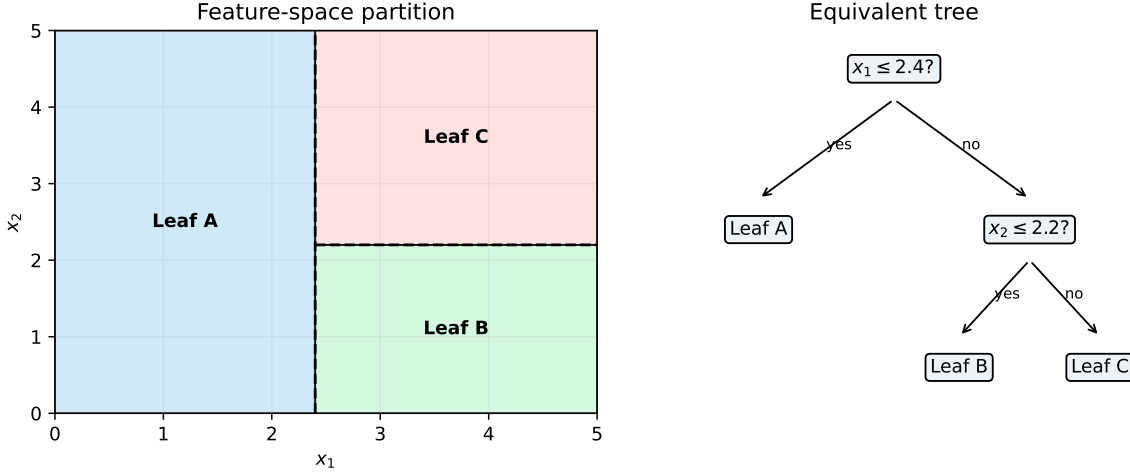


Figure 1: A shallow decision tree and the axis-aligned regions it creates.

The important visual property is axis alignment: each split cuts one feature at one threshold, so deeper trees build more refined rectangular regions.

2. Hàm Mục Tiêu và Các Độ Đo Ô Nhiễm (Impurity Metrics)

Để đánh giá chất lượng của một phép phân tách tại nút R , ta định nghĩa hàm chi phí thông qua độ ô nhiễm $L(R)$.

2.1 Đối với Bài toán Phân loại (Classification)

Gọi p_c là tỷ lệ số mẫu thuộc lớp $c \in \{1, 2, \dots, C\}$ bên trong vùng R :

$$p_c = \frac{1}{|R|} \sum_{i \in R} \mathbb{I}(y^{(i)} = c)$$

- **Độ bất định Entropy (Entropy):** Đo lường mức độ hỗn loạn của thông tin. Hàm số đạt cực đại bằng $\log_2 C$ khi dữ liệu phân phối đều và đạt cực tiểu bằng 0 khi toàn bộ mẫu thuộc về một lớp duy nhất:

$$H(R) = - \sum_{c=1}^C p_c \log_2 p_c$$

- **Chỉ số Gini (Gini Index):** Đo lường xác suất phân loại sai một mẫu dữ liệu nếu ta chọn nhãn ngẫu nhiên dựa trên phân phối của vùng đó:

$$G(R) = 1 - \sum_{c=1}^C p_c^2$$

Khi phân hoạch một nút cha R thành hai nút con R_1 và R_2 , thuật toán tham lam sẽ quét qua toàn bộ các thuộc tính j và các ngưỡng t để chọn ra phép phân tách tối đa hóa **Information Gain**:

$$\max_{j,t} \text{Gain}(R, j, t) = H(R) - \left(\frac{|R_1|}{|R|} H(R_1) + \frac{|R_2|}{|R|} H(R_2) \right)$$

2.2 Đối với Bài toán Hồi quy (Regression Trees - CART)

Đối với bài toán dự đoán giá trị liên tục, ta không sử dụng Entropy hay Gini mà tối ưu hóa bằng **Sai số bình phương trung bình (MSE)**.

- **Hàm mục tiêu ô nhiễm tại nút R :**

$$L(R) = \frac{1}{|R|} \sum_{i \in R} (y^{(i)} - \bar{y}_R)^2$$

Trong đó $\bar{y}_R$ là giá trị trung bình cộng của các nhãn mục tiêu nằm trong vùng R :

$$\bar{y}_R = \frac{1}{|R|} \sum_{i \in R} y^{(i)}$$

- **Giá trị dự đoán:** Khi một mẫu dữ liệu mới rơi vào nút lá R , giá trị dự đoán đầu ra của mô hình chính là hằng số $\bar{y}_R$.
- **Tiêu chí phân hoạch:** Thuật toán tìm kiếm thuộc tính j và ngưỡng t sao cho tổng sai số bình phương sau phân tách là nhỏ nhất:

$$\min_{j,t} \left[\sum_{i \in R_1} (y^{(i)} - \bar{y}_{R_1})^2 + \sum_{i \in R_2} (y^{(i)} - \bar{y}_{R_2})^2 \right]$$

3. Kiểm Soát Overfitting & Kỹ Thuật Cắt Tỉa Cây (Pruning)

Nếu cây quyết định được phát triển tự do, nó sẽ mọc cho đến khi tất cả các nút lá đều đồng nhất tuyệt đối ($L(R) = 0$). Hiện tượng này gây ra hiện tượng Overfitting nghiêm trọng (High Variance). Để kiểm soát, ta sử dụng thuật toán **Cost-Complexity Pruning** (Cắt tỉa chi phí - phức tạp).

Ta thiết lập hàm mục tiêu tối ưu hóa có chứa thành phần phạt dựa trên kích thước cấu trúc cây:

$$C_\alpha(T) = \sum_{m=1}^{|T|} |R_m| L(R_m) + \alpha |T|$$

Trong đó: * $|T|$ là số lượng nút lá của cây T . * $L(R_m)$ là độ ô nhiễm tại nút lá thứ m . * $\alpha \geq 0$ là siêu tham số điều khiển sự đánh đổi (trade-off) giữa độ chính xác và độ phức tạp cấu trúc. α càng lớn, thuật toán càng ép cây phải bị cắt tỉa gọn hơn.

4. Các Phương Phương Học Tập Hợp (Ensemble Methods)

4.1 Phương pháp Bagging (Bootstrap Aggregating)

- **Nguyên lý:** Xây dựng song song nhiều cây quyết định hoàn toàn độc lập. Mỗi cây được huấn luyện trên một tập dữ liệu con được tạo ra bằng phương pháp lấy mẫu lặp lại ngẫu nhiên (Bootstrap sampling) từ tập dữ liệu huấn luyện gốc.
- **Đại diện tiêu biểu - Random Forest:** Để giảm thiểu độ tương quan giữa các cây, Random Forest bổ sung một cơ chế ngẫu nhiên hóa: Tại mỗi nút phân tách, thuật toán chỉ cho phép chọn thuộc tính tối ưu từ một tập con gồm k thuộc tính được chọn ngẫu nhiên ($k < p$, thông thường $k = \sqrt{p}$).

4.2 Phương pháp Boosting & Bản chất Toán học của XGBoost

Trái ngược với Bagging, Boosting xây dựng chuỗi các cây một cách tuần tự (Sequential), cây sau tập trung sửa sai cho các cây phía trước.

XGBoost (Extreme Gradient Boosting) nâng cấp thuật toán bằng cách áp dụng **Khai triển Taylor bậc hai** để xấp xỉ hàm mất mát tổng thể tại bước lặp thứ t , giúp tìm ra cấu trúc cây tối ưu một cách cực kỳ chính xác.

Hàm mất mát tại bước lặp thứ t khi thêm cây $f_t(\mathbf{x})$ vào mô hình:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y^{(i)}, \hat{y}^{(t-1)(i)} + f_t(\mathbf{x}^{(i)})) + \Omega(f_t)$$

Áp dụng khai triển Taylor bậc hai xấp xỉ hàm $l(y, \hat{y} + \Delta\hat{y}) \approx l(y, \hat{y}) + g\Delta\hat{y} + \frac{1}{2}h(\Delta\hat{y})^2$:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y^{(i)}, \hat{y}^{(t-1)(i)}) + g_i f_t(\mathbf{x}^{(i)}) + \frac{1}{2} h_i f_t^2(\mathbf{x}^{(i)}) \right] + \Omega(f_t)$$

Trong đó g_i và h_i lần lượt là đạo hàm bậc một (Gradient) và đạo hàm bậc hai (Hessian) của hàm mất mát cũ tính tại giá trị dự đoán trước đó:

$$g_i = \frac{\partial l(y^{(i)}, \hat{y}^{(t-1)(i)})}{\partial \hat{y}^{(t-1)(i)}} \quad \text{và} \quad h_i = \frac{\partial^2 l(y^{(i)}, \hat{y}^{(t-1)(i)})}{\partial (\hat{y}^{(t-1)(i)})^2}$$

Bỏ qua các hằng số không ảnh hưởng đến quá trình tối ưu hóa, hàm mục tiêu rút gọn của XGBoost tại bước t là:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(\mathbf{x}^{(i)}) + \frac{1}{2} h_i f_t^2(\mathbf{x}^{(i)}) \right] + \gamma |T| + \frac{1}{2} \lambda \sum_{j=1}^{|T|} w_j^2$$

Thông tin đạo hàm bậc hai h_i đóng vai trò như một hệ số điều hướng độ cong của không gian lỗi, giúp XGBoost hội tụ nhanh vượt trội so với Gradient Boosting truyền thống chỉ dùng đạo hàm bậc một.

5. Quy trình chọn split trong cây quyết định

Khi đề cho bảng dữ liệu và yêu cầu chọn node gốc:

1. Tính impurity của node cha.
2. Với từng thuộc tính hoặc ngưỡng split, chia dữ liệu thành các nhánh.
3. Tính impurity của từng nhánh.
4. Tính impurity sau split:

$$Impurity_{after} = \sum_v \frac{|S_v|}{|S|} Impurity(S_v)$$

5. Tính mức giảm:

$$Gain = Impurity(S) - Impurity_{after}$$

6. Chọn split có gain lớn nhất hoặc SSE nhỏ nhất.

Bẫy thường gặp: entropy/Gini dùng cho classification; SSE/MSE dùng cho regression tree.
Đừng trộn entropy với biến mục tiêu liên tục.

6. Bảng Công thức Thi: Máy tính Phân tách Cây Quyết định

Khi đề bài hỏi một phép tách có tốt hay không, hãy tính độ ô nhiễm trước và sau khi tách.

Đại lượng	Công thức	Giá trị tốt nhất
Xác suất lớp	$p_c = \frac{\text{\#mẫu thuộc lớp } c}{ R }$	Dùng bên trong Entropy/Gini.
Entropy	$H(R) = -\sum_c p_c \log_2 p_c$	0 nghĩa là nút hoàn toàn thuần nhất.
Gini	$G(R) = 1 - \sum_c p_c^2$	0 nghĩa là nút hoàn toàn thuần nhất.
Độ ô nhiễm con có trọng số	$\frac{ R_1 }{ R } I(R_1) + \frac{ R_2 }{ R } I(R_2)$	Càng thấp càng tốt.
Information Gain	$I(R) - \left[\frac{ R_1 }{ R } I(R_1) + \frac{ R_2 }{ R } I(R_2) \right]$	Càng cao càng tốt.
Giá trị lá hồi quy	$\bar{y}_R = \frac{1}{ R } \sum_{i \in R} y_i$	Dự đoán trung bình mục tiêu trong nút lá.
Cost-complexity pruning	$C_\alpha(T) = \sum_m R_m L(R_m) + \alpha T $	α càng lớn, cây càng bị cắt gọn.

```
import numpy as np
import matplotlib.pyplot as plt

def entropy(counts):
    counts = np.array(counts, dtype=float)
    probs = counts[counts > 0] / counts.sum()
    return -np.sum(probs * np.log2(probs))

def gini(counts):
    counts = np.array(counts, dtype=float)
    probs = counts / counts.sum()
```

```

    return 1 - np.sum(probs ** 2)

parent = [5, 5]
left = [4, 1]
right = [1, 4]
n_parent = sum(parent)
weighted_entropy = (sum(left) / n_parent) * entropy(left) + (sum(right) /
    ↪ n_parent) * entropy(right)
weighted_gini = (sum(left) / n_parent) * gini(left) + (sum(right) / n_parent)
    ↪ * gini(right)

print("Entropy nút cha:", round(entropy(parent), 3))
print("Entropy con có trọng số:", round(weighted_entropy, 3))
print("Information Gain:", round(entropy(parent) - weighted_entropy, 3))
print("Gini nút cha:", round(gini(parent), 3))
print("Gini con có trọng số:", round(weighted_gini, 3))

labels = ["nút cha", "nút con"]
entropy_vals = [entropy(parent), weighted_entropy]
gini_vals = [gini(parent), weighted_gini]
x = np.arange(len(labels))
width = 0.35
plt.figure(figsize=(6, 3.5))
plt.bar(x - width / 2, entropy_vals, width, label="entropy")
plt.bar(x + width / 2, gini_vals, width, label="gini")
plt.xticks(x, labels)
plt.ylabel("độ ô nhiễm")
plt.title("Phép tách tốt làm giảm độ ô nhiễm")
plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.show()

```

```

Entropy nút cha: 1.0
Entropy con có trọng số: 0.722
Information Gain: 0.278
Gini nút cha: 0.5
Gini con có trọng số: 0.32

```

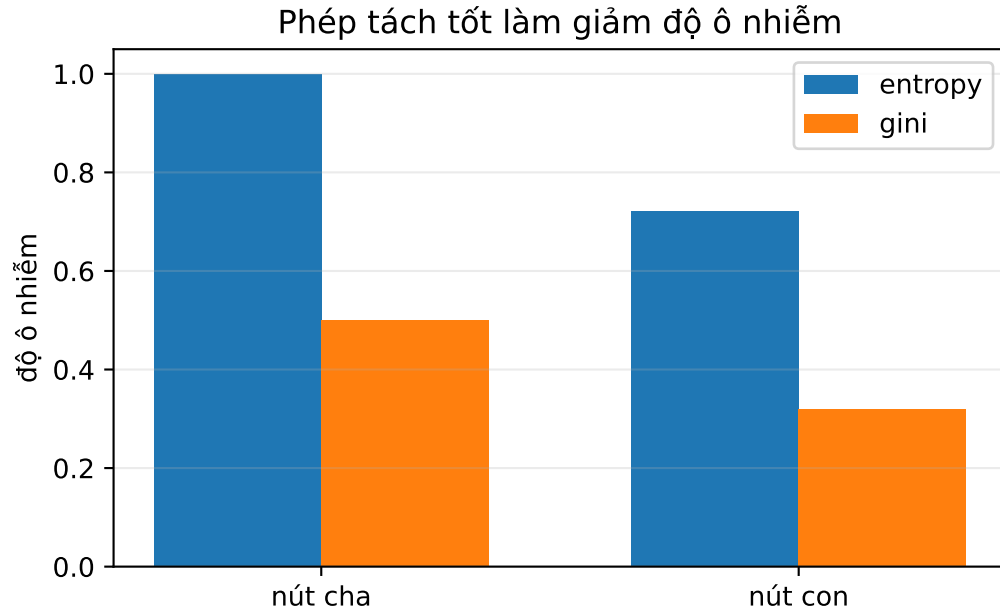


Figure 2: Information gain compares parent impurity with weighted child impurity.

Dạng bài: Information Gain từ bảng nhiều thuộc tính

Khi bảng có nhiều cột như Outlook, Temperature, Humidity, Wind, hãy tính từng thuộc tính theo cùng một khung.

Với thuộc tính A có các giá trị $v_1, \dots, v_k$:

$$Gain(S, A) = H(S) - \sum_{r=1}^k \frac{|S_{v_r}|}{|S|} H(S_{v_r})$$

Mẫu bảng phụ nên lập:

Giá trị của A	Số mẫu	Số Yes	Số No	Entropy
v_1	$ S_{v_1} $	count	count	$H(S_{v_1})$
v_2	$ S_{v_2} $	count	count	$H(S_{v_2})$

Sau đó tính weighted entropy:

$$H_{after}(A) = \sum_v \frac{|S_v|}{|S|} H(S_v)$$

và:

$$Gain(S, A) = H(S) - H_{after}(A)$$

Cách trình bày rõ ràng: tính $H(S)$ một lần ở đầu, rồi mỗi thuộc tính chỉ cần một bảng phụ và một dòng gain. Đừng trộn các count của thuộc tính này sang thuộc tính khác.

Bộ bài tập mẫu có lời giải

Đề bài. Một nút cha có 10 mẫu: 5 mẫu lớp 0 và 5 mẫu lớp 1. Một phép tách tạo ra:

- Nút trái: 4 mẫu lớp 0, 1 mẫu lớp 1.
- Nút phải: 1 mẫu lớp 0, 4 mẫu lớp 1.

Hãy tính Information Gain theo Entropy.

Lời giải từng bước.

1. Entropy nút cha:

$$H(R) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1$$

2. Entropy nút trái:

$$H(R_1) = -0.8 \log_2(0.8) - 0.2 \log_2(0.2) \approx 0.722$$

3. Entropy nút phải có cùng phân phối đảo ngược, nên:

$$H(R_2) \approx 0.722$$

4. Entropy sau tách:

$$\frac{5}{10}(0.722) + \frac{5}{10}(0.722) = 0.722$$

5. Information Gain:

$$IG = 1 - 0.722 = 0.278$$

Kết luận. Phép tách này hữu ích vì làm Entropy giảm từ 1 xuống 0.722.

Bài 2: Chọn split tốt nhất từ bộ dữ liệu 12 dòng

Đề bài. Cho bộ dữ liệu nhị phân sau. Hãy so sánh hai split: $x_1 \leq 2.5$ và $x_2 \leq 1.5$ bằng Information Gain theo Gini. Split nào tốt hơn?

dòng	x_1	x_2	y
1	1.0	1.0	0
2	1.2	1.4	0
3	1.5	2.0	0
4	2.0	1.1	0
5	2.4	1.8	1
6	2.8	1.2	1
7	3.0	2.0	1
8	3.2	2.4	1
9	3.8	1.0	1
10	4.0	2.8	1
11	4.3	1.7	1
12	4.8	2.2	1

Lời giải. Gini của một nút:

$$G(R) = 1 - p_0^2 - p_1^2$$

Information Gain theo Gini:

$$IG = G(R) - \frac{|R_L|}{|R|}G(R_L) - \frac{|R_R|}{|R|}G(R_R)$$

Nút cha có 4 mẫu lớp 0 và 8 mẫu lớp 1:

$$G(R) = 1 - \left(\frac{4}{12}\right)^2 - \left(\frac{8}{12}\right)^2 = 1 - \frac{16}{144} - \frac{64}{144} = 0.444$$

Với split $x_1 \leq 2.5$:

- Nút trái có 5 mẫu: 4 lớp 0, 1 lớp 1.
- Nút phải có 7 mẫu: 0 lớp 0, 7 lớp 1.

$$G(R_L) = 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 = 0.320$$

$$G(R_R) = 1 - 0^2 - 1^2 = 0$$

Gini sau split:

$$\frac{5}{12}(0.320) + \frac{7}{12}(0) = 0.133$$

Gain:

$$IG_{x_1} = 0.444 - 0.133 = 0.311$$

Với split $x_2 \leq 1.5$:

- Nút trái có 5 mẫu: 2 lớp 0, 3 lớp 1.
- Nút phải có 7 mẫu: 2 lớp 0, 5 lớp 1.

$$G(R_L) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.480$$

$$G(R_R) = 1 - \left(\frac{2}{7}\right)^2 - \left(\frac{5}{7}\right)^2 \approx 0.245$$

Gini sau split:

$$\frac{5}{12}(0.480) + \frac{7}{12}(0.245) \approx 0.343$$

Gain:

$$IG_{x_2} = 0.444 - 0.343 = 0.102$$

Kết luận. Chọn split có gain lớn hơn. Trong bài thi, nếu hai split cùng hợp lý trực giác, cứ để Gini/Entropy quyết định.

Bài 3: Lá hồi quy và MSE sau split

Đề bài. Với dữ liệu hồi quy 10 dòng, xét split $x \leq 5$. Mỗi lá dự đoán bằng trung bình y trong lá. Tính MSE sau split.

x	1	2	3	4	5	6	7	8	9	10
y	3	4	5	6	8	13	14	16	18	20

Lời giải.

Lá trái chứa $y = \{3, 4, 5, 6, 8\}$:

$$\hat{y}_L = \frac{3 + 4 + 5 + 6 + 8}{5} = \frac{26}{5} = 5.2$$

Sai số bình phương trong lá trái:

$$(3 - 5.2)^2 + (4 - 5.2)^2 + (5 - 5.2)^2 + (6 - 5.2)^2 + (8 - 5.2)^2 = 14.8$$

Lá phải chứa $y = \{13, 14, 16, 18, 20\}$:

$$\hat{y}_R = \frac{13 + 14 + 16 + 18 + 20}{5} = \frac{81}{5} = 16.2$$

Sai số bình phương trong lá phải:

$$(13 - 16.2)^2 + (14 - 16.2)^2 + (16 - 16.2)^2 + (18 - 16.2)^2 + (20 - 16.2)^2 = 32.8$$

Tổng SSE sau split:

$$SSE = 14.8 + 32.8 = 47.6$$

MSE sau split:

$$MSE = \frac{47.6}{10} = 4.76$$

Kết luận. Với cây hồi quy, lá không bỏ phiếu lớp mà lấy trung bình mục tiêu. Split tốt là split làm tổng sai số trong lá nhỏ đi.